

Assignment 3: OpenMP in the Wild

Bar-Ilan CS 89-3312
Parallel Systems Programming

May 20, 2026

Overview — The Story Behind This Assignment

OpenMP is the most widely-used way to add shared-memory parallelism to C programs. Adding `#pragma omp parallel for` to a loop is easy; making the result *actually faster*, *actually correct*, and *actually scaling with the number of cores* is the real engineering work.

This assignment puts you in three different situations a working parallel programmer faces every day:

1. **Data-parallel image processing** (Gaussian Blur). Lots of pixels, completely independent work per pixel, a shared read-only kernel. The textbook case for `#pragma omp parallel for`, but the choice of scheduling, loop order, and what to share vs. privatize decides whether your code scales linearly or barely at all.
2. **Iterative algorithm with reductions** (K-means Clustering). Each iteration has multiple phases (assign points, sum coordinates, update centroids) that need to stay synchronized. You will use `omp for`, `omp single`, and OpenMP's *array reductions* to avoid races without serializing your workers.
3. **Task-parallel data structure** (Concurrent Binary Search Tree). Threads will simultaneously call `insert`, `deleteNode`, `searchNode`, and `findMin` on the same tree. Your job is the synchronization *inside* the data structure — the harness will dispatch operations as `#pragma omp task`, and your locking decides whether you scale or deadlock.

For each part you are given a working **sequential** reference. Your job is to produce a **parallel** version that is **correct** (we verify against the serial output) and **scalable** (we time it with 1, 2, 4, 8, and 16 threads). A benchmark harness is provided that does the timing and the correctness check for you — you do not need to write `main.c`.

Important. Sequential code receives an automatic grade of **0**. Your `.c|` files must use OpenMP. The harness already prints the speedup numbers; if they don't grow with the thread count, you have not parallelized.

1 Part 1 — Gaussian Blur

1.1 The Story

The Gaussian blur is the most popular image-smoothing filter on the planet. Photoshop has a slider for it; every smartphone camera uses it under the hood to denoise; computer-vision pipelines use it as a preprocessing step before edge detection or feature extraction. See Adobe’s overview for context.

Mathematically, each output pixel is a weighted average of the pixels in a square neighborhood around it, where the weights come from a 2D Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

A larger σ (or a larger *radius*) means a softer blur. The provided implementation builds the Gaussian kernel once and then applies it to every interior pixel of the input image.

1.2 What You Need to Do

Take the file `guassonFilter.c` and produce a parallel version that uses OpenMP. You may add new helpers, restructure loops, hoist invariants, and pick any scheduling clause you want, **as long as the function signatures declared in `guassonFilter.h` stay identical**.

You are graded on:

1. **Correctness:** the harness compares your parallel output against a single-threaded reference run, pixel-by-pixel (interior only — the algorithm doesn’t write the border).
2. **Scalability:** the harness times you at $T \in \{1, 2, 4, 8, 16\}$ threads. We expect speedup to grow with T . A solution that doesn’t scale loses points even if it’s correct.

1.3 Files Provided (in `Part1_Gaussian/`)

- `guassonFilter.c` — the serial code you will parallelize.
- `guassonFilter.h` — types (`RGBA`, `Image`) and function prototypes.
- `main.c` — the benchmark harness. Scans `images/`, runs blur at each thread count, prints a table per image, saves blurred PNGs to `images/output/`. *You do not submit this.*
- `stb_image.h`, `stb_image_write.h` — single-header image I/O. Already `#included` for you.
- `images/` — sample photos. Re-fetch any time with `./images/fetch_images.sh`.

1.4 Input & Output

Input. Any `.jpg`, `.jpeg`, `.png`, or `.bmp` file in `images/`. The harness picks them up automatically — drop in your own to test.

Output.

- A blurred PNG for every input, in `images/output/<name>_blurred.png`.
- A per-image scalability table on stdout and in `scalability.log`, e.g.:

```

=== large.jpg (2400x1800, radius=7) ===
Threads | Time(s) | Speedup | Efficiency | Correctness
-----+-----+-----+-----+-----
1 | 4.5210 | 1.000 | 1.000 | reference (blur OK)
2 | 2.2840 | 1.979 | 0.990 | OK (matches T=1)
4 | 1.1610 | 3.894 | 0.974 | OK (matches T=1)
8 | 0.5980 | 7.561 | 0.945 | OK (matches T=1)
16 | 0.3140 | 14.398 | 0.900 | OK (matches T=1)

```

1.5 Building & Running

On the BIU servers:

```

cd Part1_Gaussian
./images/fetch_images.sh # one-time
gcc -O2 -fopenmp main.c guassonFilter.c -o gauss_bench -lm
./gauss_bench

```

1.6 Notes on Parallelization

1. The Gaussian kernel is built *once* before the pixel loop and is *read-only* during it — so it should be *shared*, not privatized.
2. Every output pixel is written exactly once by exactly one thread, so you do not need atomics or reductions *at the pixel level*.
3. Work per pixel is uniform $(2r + 1)^2$ multiply-adds. Pick a scheduling clause that matches: do you really need **dynamic** here, or does **static** do the job with less overhead?
4. Consider loop order. The image is stored row-major (`pixels[y * width + x]`); a loop order that walks down columns innermost will cache-miss every iteration. Reordering is free performance *before* you even add threading.
5. Consider `collapse(2)`, `simd`, and reductions on the inner kernel sum.

2 Part 2 — K-means Clustering

2.1 The Story

K-means is the most-taught clustering algorithm in machine learning and data mining. Given a cloud of points and a target number of clusters K , it tries to find K *centroids* (one per cluster) such that every point is closer to its own centroid than to any other. It is used for market segmentation, image compression (color quantization), document clustering, sensor data analysis, and a hundred other things.

Lloyd’s algorithm (the standard implementation) iterates two phases until convergence:

1. **Assignment:** for every point, find the nearest centroid. $O(N \cdot K)$ per iteration.
2. **Update:** for every cluster, move its centroid to the average of the points assigned to it. $O(N + K)$ per iteration.

Repeat until centroid movement falls below a tolerance, or a maximum number of iterations is reached.

2.2 What You Need to Do

Take the file `kmeans.c` and produce a parallel version using OpenMP. As with Part 1, the function signatures in `kmeans.h` are fixed, but you are free to add static helpers, hoist work, change loop structure, and pick any scheduling you want.

Both phases are parallelizable, but each in a different way:

- The *assignment* loop is embarrassingly parallel — each point’s nearest-centroid lookup is independent.
- The *update* loop has a real race condition: every point’s contribution must be added to its cluster’s accumulator (`sumX[c]`, `sumY[c]`, `counts[c]`). A naive `parallel for` would corrupt these. Think about *array reductions*, per-thread local accumulators, or atomics — and benchmark to see which is fastest.

2.3 Files Provided (in Part2_KMeans/)

- `kmeans.c` — the serial code you will parallelize.
- `kmeans.h` — types (`Point`, `PointSet`, `Centroids`) and function prototypes.
- `main.c`, `dataset.c`, `dataset.h` — the benchmark harness. Scans `inputs/`, loads each CSV, runs K-means at each thread count, prints a table per input. *You do not submit these.*
- `inputs/` — Regenerate any time with `python3 inputs/generate_inputs.py`.

2.4 Input & Output

Input format. Plain-text CSV-ish:

```
# k=8
# n=50000
123.4567 234.5678
345.6789 456.7890
...
```

Header lines (starting with #) declare K and N . Each subsequent line is one 2D point.

Output. A per-input scalability table on stdout and in `scalability.log`:

```
=== blobs_heavy_heavy_heavy_k64.csv (n=5000000, k=64, maxIters=100) ===
Threads | Time(s) | Iters | Speedup | Efficiency | Correctness
-----+-----+-----+-----+-----+-----
1 | 22.4310 | 41 | 1.000 | 1.000 | reference
2 | 11.4280 | 41 | 1.963 | 0.981 | OK (matches T=1)
4 | 5.7820 | 41 | 3.880 | 0.970 | OK (matches T=1)
8 | 2.9320 | 41 | 7.651 | 0.956 | OK (matches T=1)
16 | 1.4970 | 41 | 14.984 | 0.937 | OK (matches T=1)
```

Correctness check. The harness keeps the centroids produced at $T = 1$ as the reference, then compares each higher-thread-count run against it (sorted by coordinate, within a small tolerance to absorb FP reduction-order differences). A parallel run that drifts away from the serial result is a bug.

2.5 Building & Running

```
cd Part2_KMeans
python3 inputs/generate_inputs.py # one-time
gcc -O2 -fopenmp main.c kmeans.c dataset.c -o kmeans_bench -lm
./kmeans_bench
```

2.6 Notes on Parallelization

1. The whole iter loop can run inside a single `#pragma omp parallel` region — the thread team is built once, not `maxIters` times. Use `omp for` for the data loops and `omp single` for the small serial sections (resetting accumulators, updating centroids, checking convergence).
2. The implicit barriers at the end of `omp for` and `omp single` keep the phases in lockstep — you don't need explicit `omp barriers`.
3. For the accumulation step, prefer OpenMP's `reduction(+:sumX[:k], sumY[:k], counts[:k])` over per-thread manual buffers — the runtime tree-merges the partials for you and avoids false sharing.
4. Convergence check (`maxMovement < tolerance`) must produce the same answer in every thread. Compute it in a `omp single` block and set a shared flag.

3 Part 3 — Concurrent Binary Search Tree

3.1 The Story

So far you have written parallel *loops*. Real systems also need parallel *data structures*: shared maps, queues, indexes that many threads read and write at the same time. The textbook example is a sorted set, which is exactly what a binary search tree (BST) gives you.

In the lists recitation you saw a progression of locking strategies on a linked list — coarse-grained, fine-grained, hand-over-hand, optimistic, lazy, lock-free. In this part you will apply that thinking to a BST. The **algorithms** are the standard textbook ones for insert / delete / search / find-minimum; the interesting work is the **synchronization inside them**.

3.2 What You Need to Do

Implement the four operations in `binary_tree.c` so that **multiple threads can call them concurrently on the same tree** without corrupting it:

- `insertNode(root, x)`
- `deleteNode(root, x)`
- `searchNode(root, x)`
- `findMin(root)`

The tree operations themselves are sequential — you are *not* parallelizing a single `insertNode` call. You are making sure that when thread 1 is doing an `insertNode` and thread 2 is doing a `deleteNode` *at the same time*, the tree stays valid and both calls return correct results.

A note on what we expect (and what we don't)

Parts 1 and 2 were data-parallel problems: a fixed amount of independent work split across threads. Speedup grew almost linearly with the thread count and that was the visible win.

Part 3 is fundamentally different. A concurrent data structure has a shared, mutable state that every thread reads and writes. Every operation has to traverse from the root, so the root is a natural hot spot. As we discussed in class about parallel graph algorithms, scaling *this* class of workload is genuinely hard — impressive speedup typically requires very large problem sizes, real multi-socket hardware, and concurrency techniques (lock-free, hazard pointers, RCU, etc.) that go beyond what this assignment asks of you.

So in this part we are NOT grading you on raw speedup. What we ARE grading you on is:

1. **Correctness under concurrency:** the in-order traversal of your parallel tree must match the serial reference for every input file.
2. **No catastrophic slowdown:** the parallel run at the highest thread count should be in the same ballpark as the serial run — not 10× slower because you wrapped everything in one giant **critical section**. A small slowdown (e.g. parallel at T=16 being slightly slower than T=1) is acceptable here; a 5–10× blowup is not.

The point of this exercise is for you to *apply* the synchronization patterns from the lists recitation (coarse-grained, fine-grained, hand-over-hand, optimistic, lazy) to a tree, and *understand* why each pattern behaves the way it does. The harness still prints the scalability table so you can see what your choices cost in practice.

3.3 The API

The header is fixed — do not change it. Your job is only `binary_tree.c`.

```
#ifndef BINARY_TREE_H
#define BINARY_TREE_H

#include <stdbool.h>

typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
    /* You may add fields (e.g., an omp_lock_t per node). */
} TreeNode;

TreeNode* createNode(int data);
TreeNode* insertNode(TreeNode* root, int data);
TreeNode* deleteNode(TreeNode* root, int data);
bool searchNode(TreeNode* root, int data);
TreeNode* findMin(TreeNode* root);

void inorderTraversal(TreeNode* root); // print values, in order
void preorderTraversal(TreeNode* root);
void postorderTraversal(TreeNode* root);
void freeTree(TreeNode* root);

#endif
```

3.4 Files Provided (in Part3_BST/)

- `binary_tree.h` — the fixed API.
- `main.c` — the benchmark harness. Reads each ops file in `inputs/`, runs it serially as a reference, then runs it in parallel at each thread count, prints a table per file. *You do not submit this.*
- `inputs/` — Generate any time with `python3 inputs/generate_inputs.py`.

3.5 How the Harness Drives It (Input & Output)

The harness reads an operations file and runs it **twice**: once serially (the reference), once in parallel where the operations array is distributed across the thread team via `#pragma omp parallel for`. There are **no phase barriers**. Multiple threads simultaneously call into your four functions on the same tree; your internal locking is what keeps the tree consistent.

(We use `parallel for` rather than one `omp task` per op because per-task creation overhead is much larger than a single tree operation — with millions of tiny ops we would be benchmarking the

OpenMP runtime, not your locking. The concurrent-call semantics seen by your code are identical either way.)

After both runs finish we compare the two trees by their in-order traversals. A BST's in-order traversal is the sorted list of its values, so two BSTs containing the same set of values produce *identical* in-order output regardless of internal structure. If the two lists match, your synchronization is correct.

Why we can compare serial vs. parallel at all (determinism)

Two operations *commute* if their relative execution order does not affect the final tree state. For our four functions:

- `searchNode` and `findMin` are read-only — they always commute with anything.
- `INS x` and `INS y` commute when $x \neq y$.
- `DEL x` and `DEL y` commute when $x \neq y$.
- `INS x` and `DEL y` commute when $x \neq y$.

If every write op in the parallel section targets a *different* key, the entire stream is a set of commuting operations and the final tree state is the same no matter what order OpenMP chose to execute the tasks. That is what makes serial = parallel a valid check even though we do not synchronize the dispatch.

The provided inputs are generated under exactly that rule. Two test modes:

Mode 1: fromempty (insert-only). Tree starts empty. Stream contains only `INS`, `FIND`, and `MIN`. All `INS` keys are unique. No deletes.

```
# mode=fromempty
# n=1000000
INS 42
INS 17
FIND 42
MIN
INS 99
FIND 999
INS 5
MIN
...
```

This isolates `insertNode` under concurrent reads — the simplest case to get right, and the one your locking has to nail first.

Mode 2: buildthenattack (full mix incl. delete). The file has a build prefix marked `INIT_INS` that the harness applies *serially* to pre-populate the tree, followed by an attack section that runs in parallel:

```
# mode=buildthenattack
# n=1000000 # total attack ops (INIT_INS lines are setup, not counted)
INIT_INS 1
INIT_INS 2
INIT_INS 3
```



```

...
INIT_INS 100000
# ----- attack (dispatched as omp tasks) -----
INS 500001 # not in init tree, unique among attack INS
DEL 17 # in init tree, unique among attack DEL
FIND 1
MIN
DEL 42
INS 500002
FIND 999999
...

```

Inside the attack section every `INS` targets a key that is not already in the tree and is unique across the attack `INS` lines; every `DEL` targets a key that *is* in the initial tree and is unique across the attack `DEL` lines. By the commutativity argument above, the final tree set is

$$initial_keys \cup attack_ins_keys - attack_del_keys$$

regardless of the order in which OpenMP executes the tasks. Serial and parallel runs converge on the same tree, and we can compare them.

You, as the implementer of `binary_tree.c`, do not see the distinction between `INIT_INS` and `INS`. From your code’s perspective, the harness builds a tree somehow and then calls your four functions concurrently with no ordering guarantees. Your locking must handle every interleaving the OS scheduler can produce.

What about the return values of `FIND` / `MIN` mid-run?

The harness checks the *final tree state*, not what individual `searchNode` or `findMin` calls returned during the run. A `FIND 17` that races with `DEL 17` may legitimately return `true` or `false` depending on which task fired first — both answers are correct concurrent behavior. We trust those ops to not crash and to not corrupt the tree, and we let the final in-order check catch any structural damage.

3.6 Building & Running

```

cd Part3_BST
python3 inputs/generate_inputs.py # one-time
gcc -O2 -fopenmp main.c binary_tree.c -o bst_bench -lm
./bst_bench

```

3.7 Notes on Synchronization

1. Do not just slap one giant `critical` or `atomic` around each function — that gives catastrophic serialization (an order-of-magnitude blowup at high thread counts), and we will deduct points for it. A coarse-grained *lock* (one `omp_lock_t` at the tree level) is acceptable as a *baseline* — correct, simple, won’t deadlock.
2. Pick a strategy that matches the operation. Coarse-grained locking is easy to get right but doesn’t scale. Per-node locking with hand-over-hand traversal removes the single global bottleneck but still serializes traversals at the root. Optimistic / lazy schemes (the winning combo from the lists recitation) make reads wait-free and only lock the actual mutation point — this is the closest thing to “best of both worlds” that fits in the assignment scope.

3. `searchNode` and `findMin` are read-only — consider whether they need to lock at all, or whether they can be wait-free under your scheme.
4. Use `omp_lock_t` (declared in `<omp.h>`) for fine-grained locks. `omp_init_lock`, `omp_set_lock`, `omp_unset_lock`, `omp_destroy_lock`.
5. Free what you allocated. `freeTree` must release every `TreeNode` *and* destroy every lock you initialized in it, or you will leak.

4 Submission Guidelines

1. Work can be done individually or in pairs.
2. Submission is through the submit system.
3. Your solution must compile and run on the BIU servers without warnings (use `-Wall -Wextra` if you want to be safe).
4. In the first line of every file you submit, write your ID and full name in a comment, e.g.:
`/* 123456789 Israela Israeli */.`
5. **Sequential code is unacceptable.** A submission that does not use OpenMP receives an automatic 0.

4.1 Files to Submit

Submit a single zip file named `ass3.zip` containing exactly these files (and only these):

- `guassonFilter.c` — your parallel Gaussian blur.
- `kmeans.c` — your parallel K-means.
- `binary_tree.c` — your thread-safe BST.
- `binary_tree.h` — your h file.

Do NOT submit: the harnesses (`main.c`, `dataset.c`), the inputs (`images/`, `inputs/`), the headers we provided (`guassonFilter.h`, `kmeans.h`, `stb_*.h`), or any build artifacts.

Exact file names matter. If your zip contains `Gaussian.c` instead of `guassonFilter.c`, you receive an automatic 0. If you appeal and resubmit with the right names, 10 points are deducted from your final grade — so save yourself the hassle and double-check before uploading.

4.2 Deadline: 15/6/2026